

List α — Lean 4 / Mathlib Cheatsheet

Core namespace: `List`

Source: `Init.Data.List + Mathlib.Data.List` (Lean 4 / Mathlib4)

`List  $\alpha$` is an inductive type with two constructors: `nil : List  $\alpha$` (written `[]`) and `cons :  $\alpha \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$` (written `a :: l`). Most functions live in the `List` namespace and can be called with dot notation, e.g. `l.length`, `l.map f`.

Type & Constructors

`List  $\alpha$` the list type

`inductive List ( $\alpha : \text{Type } u$ )`

`[] / List.nil` empty list

`[] : List  $\alpha$`

`a :: l` prepend element

`cons :  $\alpha \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$`

`List.cons` sugar for nested cons

`[a, b, c] = a :: b :: c :: []`

Inspection

`l.length` number of elements

`List.length : List  $\alpha \rightarrow \mathbb{N}$`

`l.isEmpty` true iff `[]`

`List.isEmpty : List  $\alpha \rightarrow \text{Bool}$`

`l.head?` first element, option

`List.head? : List  $\alpha \rightarrow \text{Option } \alpha$`

`l.head! / l.head h` first element, panics or with proof

`List.head! : List  $\alpha \rightarrow \alpha$`

`l.getLast?` last element, option

`List.getLast? : List  $\alpha \rightarrow \text{Option } \alpha$`

`l.getLast h` last element with proof

`List.getLast : (l : List  $\alpha$ )  $\rightarrow$  l  $\neq$  []  $\rightarrow \alpha$`

`l.tail` drop first element (`[]` if empty)

`List.tail : List  $\alpha \rightarrow \text{List } \alpha$`

`l.tail?` drop first element, option

`List.tail? : List  $\alpha \rightarrow \text{Option } (\text{List } \alpha)$`

Building Lists

`l1 ++ l2` concatenate

`List.append : List  $\alpha \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$`

`l.concat a` append single element at end

`List.concat : List  $\alpha \rightarrow \alpha \rightarrow \text{List } \alpha$`

`l.reverse` reverse order

`List.reverse : List  $\alpha \rightarrow \text{List } \alpha$`

`List.replicate n a` n copies of a

`List.replicate :  $\mathbb{N} \rightarrow \alpha \rightarrow \text{List } \alpha$`

`List.range n` `[0, 1, ..., n-1]`

`List.range :  $\mathbb{N} \rightarrow \text{List } \mathbb{N}$`

`List.iota n` `[n, n-1, ..., 1]`

`List.iota :  $\mathbb{N} \rightarrow \text{List } \mathbb{N}$`

`List.join / List.flatten` concatenate list of lists

`List.join : List (List  $\alpha$ )  $\rightarrow \text{List } \alpha$`

Indexing

`l.get i` safe index (Fin)

`List.get : (l : List  $\alpha$ )  $\rightarrow$  Fin l.length  $\rightarrow \alpha$`

`l.get? n` index, returns none if out of range

`List.get? : List  $\alpha \rightarrow \mathbb{N} \rightarrow \text{Option } \alpha$`

`l.getD n d` index with default

`List.getD : List  $\alpha \rightarrow \mathbb{N} \rightarrow \alpha \rightarrow \alpha$`

`l[n]? / l[i]` subscript notation

via `GetElem` `instance`

`l.set n a` replace element at index n

`List.set : List  $\alpha \rightarrow \mathbb{N} \rightarrow \alpha \rightarrow \text{List } \alpha$`

Searching

$a \in l$	membership (decidable for DecidableEq)
<code>List.Mem</code> : $\alpha \rightarrow \text{List } \alpha \rightarrow \text{Prop}$	
<code>l.contains a</code>	Bool membership
<code>List.contains</code> : $\text{List } \alpha \rightarrow \alpha \rightarrow \text{Bool}$	
<code>l.find? p</code>	first element satisfying p
<code>List.find?</code> : $(\alpha \rightarrow \text{Bool}) \rightarrow \text{List } \alpha \rightarrow \text{Option } \alpha$	
<code>l.findIdx p</code>	index of first match (l.length if none)
<code>List.findIdx</code> : $(\alpha \rightarrow \text{Bool}) \rightarrow \text{List } \alpha \rightarrow \mathbb{N}$	
<code>l.indexOf a</code>	index of first a (DecidableEq)
<code>List.indexOf</code> : $\alpha \rightarrow \text{List } \alpha \rightarrow \mathbb{N}$	
<code>l.any p</code>	true iff some element satisfies p
<code>List.any</code> : $\text{List } \alpha \rightarrow (\alpha \rightarrow \text{Bool}) \rightarrow \text{Bool}$	
<code>l.all p</code>	true iff all elements satisfy p
<code>List.all</code> : $\text{List } \alpha \rightarrow (\alpha \rightarrow \text{Bool}) \rightarrow \text{Bool}$	
<code>l.decide? p</code>	first Some result
<code>List.findSome?</code> : $(\alpha \rightarrow \text{Option } \beta) \rightarrow \text{List } \alpha \rightarrow \text{Option } \beta$	

Transforming

<code>l.map f</code>	apply f to each element
<code>List.map</code> : $(\alpha \rightarrow \beta) \rightarrow \text{List } \alpha \rightarrow \text{List } \beta$	
<code>l.filter p</code>	keep elements where p holds
<code>List.filter</code> : $(\alpha \rightarrow \text{Bool}) \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$	
<code>l.filterMap f</code>	map + filter in one pass
<code>List.filterMap</code> : $(\alpha \rightarrow \text{Option } \beta) \rightarrow \text{List } \alpha \rightarrow \text{List } \beta$	
<code>l.flatMap f</code>	map then join
<code>List.flatMap</code> : $\text{List } \alpha \rightarrow (\alpha \rightarrow \text{List } \beta) \rightarrow \text{List } \beta$	
<code>l.bind f</code>	monad bind (alias)
<code>List.bind = List.flatMap</code>	
<code>l.partition p</code>	split into (yes, no)
<code>List.partition</code> : $(\alpha \rightarrow \text{Bool}) \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha \times \text{List } \alpha$	
<code>l.span p</code>	prefix satisfying p, rest
<code>List.span</code> : $(\alpha \rightarrow \text{Bool}) \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha \times \text{List } \alpha$	
<code>l.groupBy r</code>	group adjacent equals

`List.groupBy` : $(\alpha \rightarrow \alpha \rightarrow \text{Bool}) \rightarrow \text{List } \alpha \rightarrow \text{List } (\text{List } \alpha)$

Folding & Aggregation

<code>l.foldl f b</code>	left fold
<code>List.foldl</code> : $(\beta \rightarrow \alpha \rightarrow \beta) \rightarrow \beta \rightarrow \text{List } \alpha \rightarrow \beta$	
<code>l.foldr f b</code>	right fold
<code>List.foldr</code> : $(\alpha \rightarrow \beta \rightarrow \beta) \rightarrow \beta \rightarrow \text{List } \alpha \rightarrow \beta$	
<code>l.sum</code>	sum all elements
<code>List.sum</code> : $\text{List } \alpha \rightarrow \alpha$ [Add, Zero]	
<code>l.prod</code>	product of all elements
<code>List.prod</code> : $\text{List } \alpha \rightarrow \alpha$ [Mul, One]	
<code>l.count a</code>	occurrences of a
<code>List.count</code> : $\alpha \rightarrow \text{List } \alpha \rightarrow \mathbb{N}$	
<code>l.countP p</code>	elements satisfying p
<code>List.countP</code> : $(\alpha \rightarrow \text{Bool}) \rightarrow \text{List } \alpha \rightarrow \mathbb{N}$	
<code>l.minimum?</code>	smallest element, option
<code>List.minimum?</code> : $\text{List } \alpha \rightarrow \text{Option } \alpha$ [Min]	
<code>l.maximum?</code>	largest element, option
<code>List.maximum?</code> : $\text{List } \alpha \rightarrow \text{Option } \alpha$ [Max]	

Sublists / Slicing

<code>l.take n</code>	first n elements
<code>List.take</code> : $\mathbb{N} \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$	
<code>l.drop n</code>	all but first n
<code>List.drop</code> : $\mathbb{N} \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$	
<code>l.takeWhile p</code>	longest prefix satisfying p
<code>List.takeWhile</code> : $(\alpha \rightarrow \text{Bool}) \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$	
<code>l.dropWhile p</code>	drop while p holds
<code>List.dropWhile</code> : $(\alpha \rightarrow \text{Bool}) \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$	
<code>l.splitAt n</code>	(take n l, drop n l)
<code>List.splitAt</code> : $\mathbb{N} \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha \times \text{List } \alpha$	
<code>l.rotateLeft n</code>	rotate left by n
<code>List.rotateLeft</code> : $\text{List } \alpha \rightarrow \mathbb{N} \rightarrow \text{List } \alpha$	
<code>l.rotateRight n</code>	rotate right by n

`List.rotateRight : List α → ℕ → List α`

`l.Sublist l'` l' is a sublist (subsequence) of l

`List.Sublist : List α → List α → Prop`

Deduplication & Erase

`l.erase a` remove first occurrence of a

`List.erase : List α → α → List α`

`l.eraseIdx n` remove element at index n

`List.eraseIdx : List α → ℕ → List α`

`l.eraseP p` remove first element satisfying p

`List.eraseP : (α → Bool) → List α → List α`

`l.dedup` remove consecutive duplicates

`List.dedup : List α → List α`

`l.eraseDups` remove all duplicates (keeps first)

`List.eraseDups : List α → List α`

`l.diff l'` remove elements of l' from l

`List.diff : List α → List α → List α`

`l.union l'` $l ++$ elements of l' not in l

`List.union : List α → List α → List α`

`l.inter l'` elements of l also in l'

`List.inter : List α → List α → List α`

Zip & Enumerate

`List.zip l1 l2` pair up (shortest wins)

`List.zip : List α → List β → List (α × β)`

`List.zipWith f l1 l2` combine elementwise

`List.zipWith : (α → β → γ) → List α → List β → List γ`

`List.unzip l` split list of pairs

`List.unzip : List (α × β) → List α × List β`

`l.enum` add 0-based indices

`List.enum : List α → List (ℕ × α)`

`l.enumFrom n` add indices starting at n

`List.enumFrom : ℕ → List α → List (ℕ × α)`

Predicates & Properties

`List.Nodup l` no element appears twice

`List.Nodup : List α → Prop`

`List.Pairwise R l` R holds for all pairs $i < j$

`List.Pairwise : (α → α → Prop) → List α → Prop`

`List.Sorted r l` l is sorted by r ($=$ Pairwise r)

`List.Sorted : (α → α → Prop) → List α → Prop`

`List.Chain r a l` consecutive pairs satisfy r

`List.Chain : (α → α → Prop) → α → List α → Prop`

`List.Chain' r l` $\text{Chain}' = \text{Chain}$ on $[]$ or cons

`List.Chain' : (α → α → Prop) → List α → Prop`

Sorting

`l.mergeSort` stable merge sort

`List.mergeSort : List α → List α [Ord]`

`l.insertionSort r` insertion sort

`List.insertionSort : (α → α → Bool) → List α → List α`

`List.Sorted.merge` merge two sorted lists

`List.merge : List α → List α → List α [Ord]`

Common proof lemmas for `List` goals:

- `List.length_cons`, `List.length_append` — length arithmetic
- `List.append_nil`, `List.nil_append` — identity laws
- `List.append_assoc` — associativity of `++`
- `List.mem_cons`, `List.mem_append` — membership breakdown
- `List.map_append`, `List.map_map` — map fusion
- `List.filter_append`, `List.filter_map` — filter laws
- `List.take_append_drop` — `take n l ++ drop n l = l`
- `List.get?_map`, `List.get?_append` — index after transform
- `simp [List.mem_filter]` — simplify membership in filter
- induction `l` with `| nil => ... | cons h t ih => ...` — structural induction